

Nama : Sudrajat
Kelas : TI-24-PA2
NPM : 242310013

Praktikum 4

Latihan 1

Alur Kerja dan Modifikasi Kode

Untuk menyelesaikan Latihan 1 tanpa harus membuat navigasi antar halaman yang kompleks, saya menggunakan teknik **Conditional Rendering** dengan state management (useState).

1. **Membuat Komponen SearchBar.jsx** Komponen ini berisi TextInput untuk menerima *input* dari pengguna dan tombol *close* untuk menutup mode pencarian. Komponen ini menerima *props* *searchQuery* dan *setSearchQuery* agar teks yang diketik dapat dikelola oleh halaman utama.



```
1 import Ionicons from "@expo/vector-icons/Ionicons";
2 import { TextInput, TouchableOpacity, View } from "react-native";
3 import { color_list, styles } from "../styles/StyleApps";
4
5
6 const SearchBar = ({ onClose, searchQuery, setSearchQuery }) => {
7   return (
8     <View
9       style={{
10         flexDirection: "row",
11         alignItems: "center",
12         backgroundColor: color_list.white,
13         borderRadius: 10,
14         paddingHorizontal: 15,
15         paddingVertical: 10,
16         marginTop: 20,
17         marginBottom: 10,
18         ...styles.shadow,
19       }}
20     >
21       <Ionicons name="search-outline" size={20} color="gray" />
22       <TextInput
23         style={{ flex: 1, marginLeft: 10, fontSize: 16, color: "black" }}
24         placeholder="Search here"
25         value={searchQuery}
26         onChangeText={setSearchQuery}
27         autoFocus={true}
28       />
29       <TouchableOpacity onPress={onClose}>
30         <Ionicons name="close-circle" size={24} color="gray" />
31       </TouchableOpacity>
32     </View>
33   );
34 };
35
36 export default SearchBar;
```

2. **Memodifikasi Komponen Header.jsx** Menambahkan fungsi *onSearchClick* pada properti *onPress* di ikon kaca pembesar. Saat ikon ini ditekan, komponen akan memicu perubahan *state* di halaman utama untuk menampilkan *Search Bar*.

```
SearchBar.jsx M  Header.jsx M X  praktikum.jsx M  list_books.jsx M
components > module-latihan > Latihan-4 > components > Header.jsx > ...
You, 2 hours ago | 1 author (You)
1  import Ionicons from "@expo/vector-icons/Ionicons";
2  import { Text, TouchableOpacity, View } from "react-native";
3  import { styles } from "../styles/StyleApps";
4  You, 2 hours ago • Latihan 4
5  const Header = ({ onSearchClick }) => {
6    return (
7      <View style={styles.h_container}>
8        <View>
9          <Text style={styles.sub_title}>Good Morning 🌞</Text>
10         <Text style={styles.title}>Discover Books</Text>
11        </View>
12        <View style={{ flexDirection: "row", gap: 10 }}>
13          <TouchableOpacity
14            style={[styles.btn_icon, styles.shadow]}
15            onPress={onSearchClick}
16          >
17            <Ionicons name="search-outline" size={24} color="gray" />
18          </TouchableOpacity>
19          <TouchableOpacity style={styles.btn_icon}>
20            <Ionicons name="notifications-outline" size={24} color="gray" />
21          </TouchableOpacity>
22        </View>
23      </View>
24    );
25  };
26  export default Header;
```

3. **Memodifikasi Halaman Utama praktikum.jsx** Menggunakan `isSearchActive` untuk mengatur logika tampilan. Jika bernilai `true`, maka aplikasi hanya merender `SearchBar` dan daftar buku. Halaman utama juga menyimpan logika `.filter()` untuk mencocokkan input teks dari `SearchBar` dengan judul buku yang ada di dalam `ListBook`.

```

export default function HomeScreen() {
  const lastBook = ListBook[ListBook.length - 1];
  const [isSearchActive, setIsSearchActive] = useState(false);
  const [searchQuery, setSearchQuery] = useState("");
  const filteredBooks = ListBook.filter((book) =>
    book.title.toLowerCase().includes(searchQuery.toLowerCase())
  );

  return (
    <SafeAreaView style={styles.container}>
      <StatusBar barStyle="dark-content" />

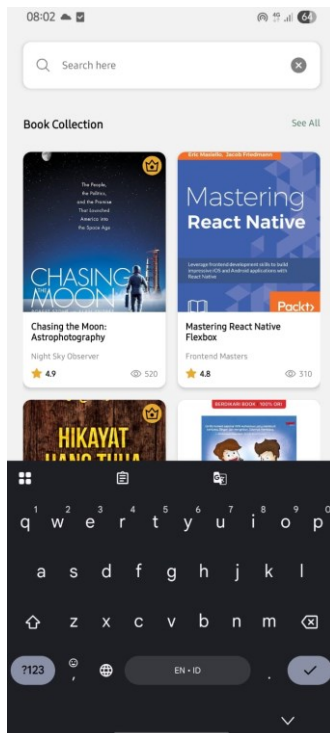
      {isSearchActive ? (
        <>
          <SearchBar
            onClose={() => {
              setIsSearchActive(false);
              setSearchQuery("");
            }}
            searchQuery={searchQuery}
            setSearchQuery={setSearchQuery}
          />
          <ScrollView showsVerticalScrollIndicator={false}>
            <BookCollections books={filteredBooks} />
          </ScrollView>
        </>
      ) : (
        <>
          <Header onSearchClick={() => setIsSearchActive(true)} />
          <ScrollView
            showsVerticalScrollIndicator={false}
            contentContainerStyle={{ paddingBottom: 20 }}
          >
            <View style={{ flex: 1 }}>
              <CTABook book={lastBook} />
              <Categoriesnav />
              <BookCollections books={ListBook} />
            </View>

            <View>
              <Text style={{ color: color_list.green }}>
                &copy; 2026 Sudrajat
              </Text>
            </View>
          </ScrollView>
        </>
      )}
    </SafeAreaView>
  );
}

```

Hasil Tampilan (UI)

Berikut adalah tampilan aplikasi ketika tombol *search* pada *header* ditekan dan fitur pencarian aktif:



Latihan 2

Alur Kerja dan Modifikasi Kode

Praktik *reusable component* diterapkan dengan memanggil komponen `<BookCollection/>` (yang sudah berisi rancangan UI *Grid* dan *Card* buku) ke dalam blok tampilan pencarian di halaman utama.

Komponen ini dirancang secara dinamis sehingga menerima *props* berupa data array (dalam hal ini `filteredBooks`). Dengan memanggil satu baris tag XML `<BookCollection books={filteredBooks} />`, aplikasi secara otomatis merender daftar buku sesuai dengan hasil pencarian yang diketik oleh pengguna, tanpa perlu membangun ulang elemen gambar, teks judul, author, dan rating dari nol.

```
contentContainerStyle={{ paddingBottom: 28 }}
>
<View style={{ flex: 1 }}>
  <CTABook book={lastBook} />
  <Categoriesnav />
  <BookCollections books={ListBook} />
</View>
```

Hasil Tampilan (UI)

Tampilan koleksi buku yang dirender ulang pada mode pencarian beroperasi dengan sempurna, menampilkan format tata letak yang konsisten dengan halaman *Home* utama.

